FairSplit: Mitigating Bias in Graph Neural Networks through Sensitivity-based Edge Partitioning

Indranil Ojha
Indian Statistical Institute
Kolkata, India
ojhaindranil@gmail.com

Kushal Bose
Indian Statistical Institute
Kolkata, India
kushalbose92@gmail.com

Swagatam Das
Indian Statistical Institute
Kolkata, India
swagatamdas19@yahoo.co.in

Abstract

Fairness in machine learning has become increasingly crucial, particularly in graph-based models where biased representations can reinforce societal inequalities. Traditional fairness-aware learning methods on graphs focus on graph rewiring, debiasing node embeddings, adversarial learning, and additional fairness constraints. However, these approaches often struggle to balance fairness and task performance. We propose a novel edge partitioning strategy that creates two distinct subgraphs, maintaining a balance between bias and diversity. We categorize edges as homophilic or heterophilic depending on the sensitive attribute of the corresponding node pairs. An edge is *s-homophilic* if it joins two nodes with the same sensitivity value, otherwise *s-heterophilic*. The partition splits the input graph into two subgraphs, both containing all nodes, one with only s-homophilic edges and the other with s-heterophilic ones. Using a Graph Neural Network (GNN), we obtain independent node representations from both graphs, which are then aggregated into a unified node embedding. To enforce fairness, we jointly optimize a primary task loss and a fairness loss, ensuring predictive accuracy and bias mitigation. We evaluate our approach on three benchmark datasets and find that it achieves improved fairness metrics while maintaining accuracy comparable to that of existing state-of-the-art methods.

CCS Concepts

• **Computing methodologies** → **Machine learning**.

Keywords

Fairness, Edge splitting, s-homophily, GNN, Aggregation, Sensitivity attributes

ACM Reference Format:
Indranil Ojha, Kushal Bose, and Swagatam Das. 2025. FairSplit: Mitigating Bias in Graph Neural Networks through Sensitivity-based Edge Partitioning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3746252.3760951

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '25, Seoul, Republic of Korea
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3760951

1 Introduction

Fairness in machine learning is emerging as a significant area of focus, especially for graph-based models applied in high-stakes domains such as criminal justice, hiring, and finance, where decisions carry far-reaching consequences. Graph Neural Networks (GNNs), which leverage graph structures to enhance predictive accuracy, have been shown to propagate and even amplify biases inherent in real-world data [3, 9], leading to systemically adverse outcomes for certain protected groups and reinforcing existing inequalities.

Unlike other modalities, fairness challenges in graph data arise not only from biased node attributes but also from structural properties like homophily and skewed degree distributions, which can entrench unfairness during message passing [19, 31]. To address these challenges, researchers have proposed fairness-aware methods across the machine learning pipeline, spanning pre-processing, in-processing, and post-processing interventions.

Related Works Pre-processing approaches aim to mitigate structural bias before training by modifying the graph or features. Graph rewiring methods adjust edges to weaken correlations with sensitive attributes [3, 19, 21], while feature-level interventions desensitize input attributes [2, 7, 29]. In-processing strategies embed fairness constraints directly into training, using fairness-aware regularization [23, 32] or adversarial learning to obscure sensitive information in node embeddings [22, 27]. Post-processing methods adjust predictions to enforce fairness without altering the model internals [10, 14, 15, 18]. Additionally, graph augmentation has emerged as a fairness tool, leveraging edge perturbations [5, 12, 24] or contrastive learning to encourage fair representations [4, 25, 30].

Limitations of Current Approaches Despite steady progress, the prevailing works face a slew of challenges. As we know, sensitive attributes are deeply woven into graph topology, so even modest structural biases, such as homophily or skewed degree distributions, can be amplified by message passing. Current strategies each carry shortcomings: (1) edge rewiring or dropout can erase semantically important connections, affecting the downstream tasks; (2) feature masking and adversarial debiasing often presume complete, publicly available sensitive labels, an assumption rarely met in practice; (3) fairness-regularized objectives may destabilize training and produce opaque trade-offs between accuracy and equity; and (4) post-hoc calibration lacks theoretical guarantees and can break when the graph evolves. These gaps underscore the need for methods that can retain the feature semantics and graph structure, and improve fairness with almost no impact on the performance of the downstream task.

In this work, we propose FairSplit, a novel fairness-aware approach that leverages homophily based edge-splitting to generate

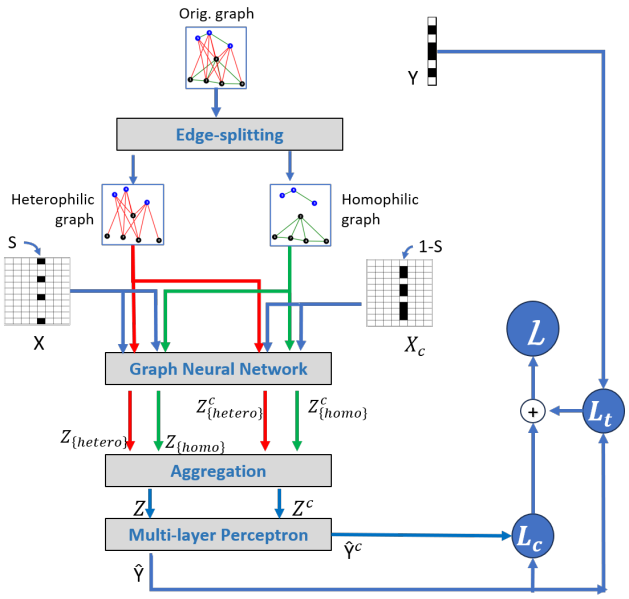

Figure 1: The complete workflow of FairSplit is illustrated. The edges are partitioned according to the sensitive attributes of adjacent node features, and two separate graphs are obtained. A shared GNN is employed to learn two node embeddings and are adaptively combined to generate balanced and unbiased representations. Additionally, task loss and fairness loss are added to enable fair learning.

diverse node representations. Our method is inspired by graph augmentation strategies, but instead of randomly perturbing edges, we introduce a streamlined edge-splitting strategy based on the sensitive attributes of the respective node pairs. In the fairness context, each node feature inherently comes with a sensitive attribute. If adjacent nodes share identical sensitivity values, then the edge between them is assigned an edge attribute as 1, otherwise 0. We call an edge *s-homophilic* if the edge attribute is 1, otherwise it will be tagged as *s-heterophilic*.

Based on this, we construct two separate graphs that contain all nodes of the original graph, but either contain homophilic edges or heterophilic edges. We apply a pre-defined GNN model shared on this pair of graphs to learn node embeddings that retain bias from the homophilic subgraph and introduce diversity from the heterophilic subgraph. Furthermore, we design an adaptive mechanism, learnable and at feature level, to combine the two representations, maintaining a balance between bias and diversity. Subsequently, these combined node embedding are fed into a dense network to project into a relevant space for the downstream task. Additionally, we extend our algorithm into another variant named FairSplit2, where two different GNNs are employed to extract node representations from homophilic and heterophilic subgraphs individually. To balance fairness and task performance, we optimize a combined loss function that incorporates both primary task loss and a fairness loss to ensure unbiased representations.

Contribution Our contributions are: (1) We propose a novel edge-splitting framework using s-homophily of the edges. It partitions the graph into two structurally distinct subgraphs, reducing bias propagation in GNNs. We argue that this helps us separately process the connections that are community-forming (a hindrance to fairness) and those that are not. (2) We introduce a dual-representation learning approach, where each node obtains separate embeddings from both subgraphs, which are then aggregated to a unified and fair representation. One aggregation method has been suggested by us, but depending on the task, other methods can also be used. (3) Our method is evaluated on multiple datasets and GNN architectures, demonstrating improved fairness while maintaining strong predictive performance.

2 Proposed Methodology

2.1 Notations and Preliminaries

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph where $\mathcal{V}$ is the set of nodes with $|\mathcal{V}| = n$, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Let $A \in \mathbb{R}^{n \times n}$ be the adjacency matrix of the graph $\mathcal{G}$, without self-loops. Also, let $X \in \mathbb{R}^{n \times m}$ be the feature matrix where m is the feature dimension, with each feature vector normalized, and x_i be the feature vector of node i. Also, $\mathcal{Y}$ denotes the set of all class labels.

Let s be a binary sensitive attribute (like race or gender). Now, we have the following definition.

Definition 2.1 (s-homophily). An edge $e = (i, j)$ is termed as ***s-homophilic*** if the sensitive attributes s_i and s_j for respectively nodes i and j are equal, i.e. $s_i = s_j$. Otherwise, it is called ***s-heterophilic***. The *s-homophily* of a graph is defined as the fraction of s-homophilic edges to the total edges present in the graph.

2.2 Our Method: *FairSplit*

In this section, we introduce our approach with the following steps,

[1] Edge splitting. The edges of the input graph are tagged with s-homophilic and s-heterophilic categories based on the definition 2.1. This homophily strictly differs from the standard label-centric homophily [17, 33]. After categorizing edges into two well-defined groups, we generate two subgraphs where one will now only retain homophilic edges and the other one will contain all heterophilic edges. More specifically, we divide the whole edge set $\mathcal{E}$ into disjoint partitions of homophilic edge set $\mathcal{E}_{homo}$ and heterophilic edge set $\mathcal{E}_{hetero}$ that is $\mathcal{E}_{\text{homo}} \cup \mathcal{E}_{\text{hetero}} = \mathcal{E}$, and $\mathcal{E}_{\text{homo}} \cap \mathcal{E}_{\text{hetero}} = \phi$. The edge partitioning divide the original graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ into two graphs as $\mathcal{G}_{homo} = (\mathcal{V}, \mathcal{E}_{homo})$ and $\mathcal{G}_{hetero} = (\mathcal{V}, \mathcal{E}_{hetero})$. This disjoint partition of the edges offers two distinct views of the entire graph. Importantly, the homophilic graph captures the inductive bias, and its heterophilic counterparts induce diversity. Refer to Figure 2 for the vivid illustration of the edge-splitting process and the resulting two disjoint views of graphs.

[2] Learning Node Representation. Two distinct graphs are fed into a pre-defined GNN architecture to learn node representations. The twin graphs learns from the same GNN models sharing the learnable parameters Θ_{GNN}. The node embeddings are learned in the following way,

$$\begin{aligned} Z_{\text{homo}} &= \text{GNN}(X, \mathcal{E}_{homo} \mid \Theta_{GNN}), \\ Z_{\text{hetero}} &= \text{GNN}(X, \mathcal{E}_{hetero} \mid \Theta_{GNN}) \end{aligned} \tag{1}$$

where $Z_{homo}, Z_{hetero} \in \mathbb{R}^{n \times h}$,are respectively the learned node representation from $\mathcal{G}_{\text{homo}}$ and $\mathcal{G}_{\text{hetero}}$ with h as the embedding dimension. The purpose of employing the GNN is to obtain graph-specific node embeddings.

[3] Adaptive Aggregation. We combine two representations Z_{homo} and Z_{hetero} by introducing learnable weights. The adaptive aggregation enables the final representation to be aligned with the downstream task. We employed two learnable weights $W_{\text{homo}}, W_{\text{hetero}} \in \mathbb{R}^{h \times h}$ to adaptively balance between homophilic and heterophilic representations implemented in the following way,

$$Z = (Z_{\text{homo}} W_{\text{homo}}) + (Z_{hetero} W_{hetero}) \,. \tag{2}$$

The adaptive approach ensures that each attribute gets a different weight during aggregation.

[4] Prediction. The adaptive aggregation facilitates the learning of compact representations. In this context, as our target is to perform node classification, we feed Z to a multi-layered perceptron (MLP) to generate soft labels as presented in the following way,

$$\hat{Y} = \text{MLP}(Z \mid \Theta_{MLP}), \tag{3}$$

where Θ_{MLP} denotes the parameterized weight matrices.

Another variant (FairSplit2) In the FairSplit, a single GNN processes both homophilic and heterophilic subgraphs, enabling weight sharing. An alternative architecture, FairSplit2, uses two independent GNNs, one per subgraph, to potentially explore their structural connections. We can use a simple GNN, such as GCN [20] or GraphSage [16], to process a homophilic graph, and a more sophisticated one, such as GPRGNN [6] or H2GCN [33], that works better for heterophilic graphs, for the heterophilic counterparts. FairSplit2 is assumed to be more capable of learning enriching node embeddings, but this increases the number of model parameters.

Training & Loss Functions. There are two independent objectives for the algorithm. One is to minimize the loss for the main task which is, in this case, node classification. The other one is maximizing the fairness of the algorithm. For the main task of node classification, we used cross-entropy loss between the ground truth and the prediction.

$$\mathcal{L}_t = \text{CrossEntropy}(Y, \hat{Y}). \tag{4}$$

For the fairness training, we use counterfactual loss. If X is the feature set, let X_c be the feature set with all features the same as X, only the sensitive feature flipped. Let $\mathcal{M}$ be our model discussed above. Then our counterfactual loss can be computed as:

$$\hat{Y} = \mathcal{M}(X, \mathcal{E}) \quad \hat{Y}_c = \mathcal{M}(X_c, \mathcal{E}) \quad \mathcal{L}_c = \text{CrossEntropy}(\hat{Y}, \hat{Y}_c). \tag{5}$$

Finally, we combine the two losses simply by summing them up.

$$\mathcal{L} = \mathcal{L}_t + \mathcal{L}_c. \tag{6}$$

2.3 Complexity Analysis

The time complexity of FairSplit depends on the underlying GNN architectures, with two additional steps: aggregation and prediction. For a GCN layer, the complexity is $O(em + nmh)$; for GraphSAGE, $O(nsm)$ with s being the number of sampled neighbors; and for APPNP, $O(ke + nmh)$ for k iterations. Beyond the GNN layers, our method introduces an aggregation step with complexity $O(nh^2)$ and a prediction step with $O(nhc)$, where c is the number of classes.

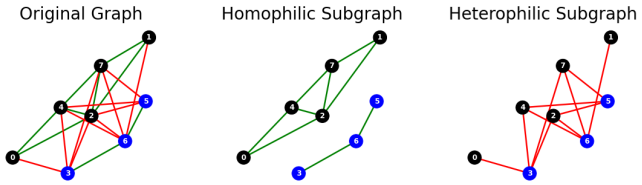

Figure 2: The elucidation of the dual view of the input graph based on s-homophily properties. Each color represents sensitive attributes. Note that subgraphs contain only either s-homophilic or s-heterophilic edges.

Here, n and e denote the number of nodes and edges, m is the input feature dimension, h is the hidden dimension, and c is the number of classes. Since $h < m$ and c is comparatively smaller, the additional aggregation and prediction steps do not increase the overall complexity for GCN and APPNP. In summary, our approach incurs minimal computational overhead while maintaining scalability comparable to the base GNN.

3 Experiments

Datasets We performed our experiments on three datasets - *German Credit*, *Recidivism*, and *Credit Default*. In the *German Credit* dataset, each node denotes an individual associated with a specific German bank. Edges indicate similarities in their credit account profiles. The classification task involves predicting whether an individual poses a high or low credit risk, with gender serving as the sensitive attribute. In *Credit Default* dataset, each node represents an individual using a form of credit. Connections between individuals are based on similarities in spending patterns and payment behavior. The classification objective is to predict whether a person is likely to default on their credit card payment, with age considered as the sensitive attribute. In the *Recidivism* dataset, each node corresponds to a defendant who was granted bail within the U.S. state court system between 1990 and 2009. Edges reflect similarities in criminal history and offense types. The goal of the classification task is to predict whether a defendant should be granted bail—interpreted as being unlikely to commit a violent crime if released—or denied bail, indicating a higher risk of violent re-offense. Race is treated as a sensitive attribute in this analysis.

Baselines We considered three widely adopted base models like GCN [20], GraphSage [16], and APPNP [13]. For each combination of dataset and model, we compare our approach with five state-of-the-art fairness algorithms for GNNs like FairGNN [8], NIFTY [1], FairVGNN [26], EDITS [11], and SRGNN [28]. Our codebase is available at https://github.com/kushalbose92/FairSplit.

Evaluation Metrics We evaluate our approach using four evaluation metrics. Performance of the main task of node classification is measured using accuracy and F1-score. Fairness is monitored with *Statistical Parity* (Δ_{SP}) and *Equal Opportunity* (Δ_{EO}). Δ_{SP} measures whether prediction outcomes are independent of the sensitive attribute, while Δ_{EO} evaluates fairness conditioned on the ground truth being positive.

Hyperparameter Tuning Besides standard hyperparameters such as learning rate, weight decay, or dropout, our proposed method

Table 1: Performance of FairSplit paired with GCN, APPNP, and GraphSage is presented. The benchmark results used for comparison are cited from [28]. The best and second-best results are marked in green and blue colors.

Model	Recidivism				Credit Default				German Credit			
	$ACC(\uparrow)$	$F1(\uparrow)$	$\Delta_{SP}(\downarrow)$	$\Delta_{EO}(\downarrow)$	$ACC(\uparrow)$	$F1(\uparrow)$	$\Delta_{SP}(\downarrow)$	$\Delta_{EO}(\downarrow)$	$ACC(\uparrow)$	$F1(\uparrow)$	$\Delta_{SP}(\downarrow)$	$\Delta_{EO}(\downarrow)$
GCN	83.19 ± 0.42	76.52 ± 0.48	7.36 ± 0.22	5.29 ± 0.43	73.62 ± 0.06	81.88 ± 0.06	12.93 ± 0.26	10.65 ± 0.18	71.84 ± 1.53	79.86 ± 31.63	38.96 ± 5.83	31.18 ± 6.32
FairGCN	83.40 ± 1.19	77.73 ± 1.31	8.09 ± 0.84	5.77 ± 1.10	73.27 ± 1.16	81.73 ± 1.15	14.59 ± 4.70	12.56 ± 5.06	60.24 ± 12.09	61.52 ± 18.52	24.81 ± 9.51	20.29 ± 7.13
NI-GCN	72.60 ± 15.36	75.04 ± 3.65	4.26 ± 0.79	3.26 ± 1.24	72.05 ± 2.48	81.74 ± 0.04	11.74 ± 0.07	9.47 ± 0.08	66.56 ± 3.90	82.34 ± 0.13	25.12 ± 2.95	21.10 ± 2.66
FV-GCN	85.47 ± 0.47	79.38 ± 0.25	5.66 ± 0.68	4.29 ± 1.15	78.44 ± 0.41	87.57 ± 0.09	3.37 ± 2.57	1.59 ± 1.30	70.08 ± 0.85	81.45 ± 0.55	4.71 ± 4.37	4.29 ± 2.78
ED-GCN	85.66 ± 0.17	79.90 ± 0.26	5.74 ± 0.19	3.59 ± 0.26	81.12 ± 4.28	77.73 ± 2.35	8.64 ± 2.57	6.30 ± 2.32	63.84 ± 6.31	73.61 ± 6.22	6.90 ± 4.80	6.28 ± 2.72
SR-GCN	90.99 ± 0.38	87.28 ± 0.14	0.72 ± 0.66	1.26 ± 0.48	78.11 ± 0.21	87.54 ± 0.17	1.67 ± 1.19	0.98 ± 0.51	70.24 ± 0.41	82.40 ± 0.23	1.99 ± 1.53	0.90 ± 0.79
FairSplit-GCN (Ours)	81.75 ± 0.22	72.11 ± 0.17	7.21 ± 0.44	1.55 ± 0.47	79.42 ± 0.16	87.98 ± 0.05	0.88 ± 0.41	0.86 ± 0.35	71.30 ± 1.06	82.93 ± 0.58	5.51 ± 4.31	0.21 ± 0.67
APPNP	80.08 ± 0.27	76.61 ± 0.32	7.37 ± 0.26	4.34 ± 0.52	74.73 ± 0.30	82.83 ± 0.20	15.00 ± 0.55	12.85 ± 0.60	71.29 ± 2.41	79.19 ± 2.70	28.61 ± 11.66	21.40 ± 8.20
FairAPPNP	82.55 ± 0.82	76.57 ± 0.93	6.81 ± 1.51	5.37 ± 1.35	73.16 ± 1.02	81.58 ± 0.78	14.43 ± 3.17	11.88 ± 3.27	66.08 ± 5.93	73.76 ± 9.37	18.23 ± 8.53	13.49 ± 7.17
NI-APPNP	81.24 ± 2.91	68.73 ± 6.20	4.01 ± 0.91	3.21 ± 1.41	72.82 ± 0.87	81.74 ± 0.04	9.83 ± 4.00	9.51 ± 0.09	64.32 ± 9.39	81.81 ± 1.25	8.99 ± 5.04	6.22 ± 2.64
FV-APPNP	80.63 ± 0.87	73.72 ± 2.05	7.87 ± 0.71	7.27 ± 1.84	78.44 ± 0.49	87.69 ± 0.12	2.40 ± 2.54	1.24 ± 1.33	69.28 ± 0.47	81.80 ± 0.38	2.17 ± 2.18	2.71 ± 2.34
ED-APPNP	82.82 ± 1.17	78.04 ± 1.18	5.92 ± 0.86	3.26 ± 1.63	79.61 ± 2.78	77.71 ± 2.39	8.75 ± 2.75	5.81 ± 2.76	70.56 ± 0.90	79.54 ± 1.73	12.05 ± 8.67	6.56 ± 4.90
SR-APPNP	88.56 ± 0.89	83.11 ± 1.60	0.85 ± 0.17	3.37 ± 1.23	77.37 ± 1.06	86.73 ± 1.30	1.12 ± 0.88	0.70 ± 0.79	70.00 ± 0.05	82.32 ± 0.07	0.51 ± 1.02	0.71 ± 1.42
FairSplit-APPNP (Ours)	91.63 ± 6.94	91.87 ± 5.36	6.84 ± 1.74	1.57 ± 1.12	80.13 ± 0.39	88.15 ± 0.12	1.67 ± 1.20	0.58 ± 0.52	71.90 ± 1.45	82.77 ± 0.92	7.29 ± 3.44	2.19 ± 1.82
Sage	85.44 ± 2.24	80.74 ± 0.92	6.50 ± 0.76	5.60 ± 1.25	74.20 ± 0.60	82.45 ± 0.52	16.35 ± 2.36	14.12 ± 2.64	71.44 ± 1.65	80.08 ± 1.64	26.48 ± 6.20	22.79 ± 8.36
FairSAGE	83.46 ± 2.70	78.77 ± 1.99	4.88 ± 2.13	4.77 ± 1.48	72.44 ± 2.28	80.95 ± 2.03	20.87 ± 9.69	19.37 ± 11.00	70.72 ± 1.65	79.97 ± 2.62	4.21 ± 2.34	5.36 ± 2.07
NI-Sage	66.12 ± 22.69	76.17 ± 10.13	4.86 ± 1.99	4.18 ± 0.88	69.80 ± 5.73	82.20 ± 0.70	12.71 ± 1.77	10.43 ± 1.58	66.24 ± 4.12	78.27 ± 1.25	8.03 ± 7.19	4.40 ± 4.18
FV-Sage	87.61 ± 1.30	82.67 ± 0.87	3.49 ± 1.74	2.42 ± 1.29	76.06 ± 4.37	84.43 ± 4.23	9.06 ± 3.63	5.90 ± 3.54	69.60 ± 1.13	81.33 ± 0.55	2.50 ± 3.01	1.26 ± 1.07
ED-Sage	85.10 ± 2.62	80.82 ± 2.03	7.67 ± 0.35	7.42 ± 0.53	83.73 ± 0.73	76.93 ± 0.89	7.28 ± 0.49	5.09 ± 0.78	70.72 ± 1.65	79.97 ± 2.62	4.21 ± 2.34	5.36 ± 2.07
SR-Sage	88.09 ± 0.59	83.29 ± 0.87	0.81 ± 0.66	0.65 ± 0.58	72.29 ± 3.65	81.23 ± 3.86	3.15 ± 1.67	1.07 ± 1.22	70.56 ± 0.78	82.59 ± 0.32	0.64 ± 0.85	0.34 ± 0.67
FairSplit-Sage (Ours)	98.45 ± 0.37	95.29 ± 5.34	7.95 ± 1.26	1.38 ± 0.71	81.48 ± 0.31	88.48 ± 0.09	2.09 ± 0.01	1.54 ± 1.12	68.90 ± 3.21	79.38 ± 2.48	12.29 ± 5.82	6.39 ± 6.75

needs an embedding dimension h that has to be chosen. We used grid-search for learning rate from (0.01, 0.001, 0.0001), weight decay from (0.001, 0.0001, 0.00001), and dropout from (0.2, 0.5). We kept $h = 32$ all along.

Sampling for Large graphs: The training of large-scale graphs is facilitated by extracting subgraphs to overcome memory constraints, where the maximum number of nodes per subgraph is data-specific. The German Credit contains 1000 nodes, hence no sampling was required. For the Recidivism and Credit Default datasets, 10000 and 15000 nodes were chosen, respectively, creating two subgraphs for each. Average performance over five runs on each subgraph has been reported.

Results & Discussions We present the quantitative analysis in the Table 1. A closer observation reveals that FairSplit attains overall impressive performances across three benchmark datasets. We evaluated a total of 36 scenarios (3 datasets, 3 GNNs, 4 metrics), and among them, FairSplit outperformed in 17 cases and ended as runner-up in 8 cases, which signifies the effectiveness. Furthermore, we have estimated the average rankings among 7 contender algorithms separately on each dataset and noticed that FairSplit ranks at the top for the Credit Default dataset (average rank 1.33), second for Recidivism (3.25), and third for German Credit (3.17). Credit Default has the highest community formation among the three datasets, with a s-homophily ratio 0.86 compared to the same for Recidivism (0.51) and German Credit (0.67). The higher s-homophily values indicate the higher impact of bias embedded in the dataset. Additionally, Credit Default is the largest graph compared to other datasets. The performance of FairSplit on Credit Default underscores the efficacy of our framework in alleviating bias. Notably, the overall consistent performance in terms of the four metrics highlights its potential for enforcing fairness.

Performance Analysis of FairSplit2 We experimented on FairSplit2 to evaluate the utility of employing two individual GNN architectures. We considered three homophilic models, GCN, APPNP, and GraphSage and one heterophilic model, MixHop. We applied three distinct combinations with each homophilic model and the heterophilic model. The homophilic and heterophilic models are employed, respectively, to learn from homophilic and heterophilic subgraphs. The application of individual GNNs will increase the parameter complexity of the model and is poised to be more effective than when only a shared GNN is used.

Table 2: A comparative study between FairSplit and FairSplit2 is demonstrated with the three backbones GCN, APPNP, and GraphSage. FairSplit2 exhibited competitive performance compared to FairSplit, thereby underlining its importance. The best results are marked in green colors.

Model	Credit Default			
	$ACC(\uparrow)$	$F1(\uparrow)$	$\Delta_{SP}(\downarrow)$	$\Delta_{EO}(\downarrow)$
FairSplit-GCN	79.42 ± 0.16	87.98 ± 0.05	0.88 ± 0.41	0.86 ± 0.35
FairSplit2-GCN-MixHop	79.77 ± 0.44	88.23 ± 1.84	1.33 ± 0.46	0.29 ± 0.23
FairSplit-APPNP	80.13 ± 0.39	88.15 ± 0.12	1.67 ± 1.20	0.58 ± 0.52
FairSplit2-APPNP-MixHop	80.70 ± 1.29	88.33 ± 0.33	2.18 ± 1.40	0.61 ± 0.42
FairSplit-Sage	81.48 ± 0.31	88.48 ± 0.09	2.09 ± 0.01	1.54 ± 1.12
FairSplit2-Sage-MixHop	79.30 ± 1.21	87.85 ± 0.22	1.59 ± 1.34	1.06 ± 0.88

We applied both FairSplit and FairSplit2 to the Credit Default graph with various combinations, and the quantitative results are presented in Table 2. A close observation reveals that the performance of FairSplit2 compared to FairSplit improved in some of the scenarios. Therefore, combining both variants offers improved performance, complementing each other.

4 Conclusion and Future Works

We introduced a fairness-aware graph learning framework that splits the graph into homophilic and heterophilic subgraphs, enabling separate representation learning from each structure, and an adaptive aggregation for final predictions. Our approach improves fairness metrics across benchmarks with minimal accuracy loss, offering a flexible and interpretable solution. Future work can include extending the method to dynamic graphs, and exploring tighter integration of fairness objectives into the aggregation process can be a potential research direction.

5 GenAI Usage Disclosure

No major usage of GenAI tools is involved in writing the manuscript. Some linguistic errors are rectified by standard online tools like Grammarly.

References

[1] Chirag Agarwal, Himabindu Lakkaraju, and Marinka Zitnik. 2021. Towards a unified framework for fair and stable graph representation learning. In *Uncertainty in artificial intelligence*. PMLR, 2114–2124.

[2] Avishek Bose and William Hamilton. 2019. Compositional fairness constraints for graph embeddings. In *International conference on machine learning*. PMLR, 715–724.

[3] April Chen, Ryan A Rossi, Namyong Park, Puja Trivedi, Yu Wang, Tong Yu, Sungchul Kim, Franck Dernoncourt, and Nesreen K Ahmed. 2024. Fairness-aware graph neural networks: A survey. *ACM Transactions on Knowledge Discovery from Data* 18, 6 (2024), 1–23.

[4] Wei Chen, Meng Yuan, Zhao Zhang, Ruobing Xie, Fuzhen Zhuang, Deqing Wang, and Rui Liu. 2024. FairDgcl: Fairness-aware Recommendation with Dynamic Graph Contrastive Learning. *arXiv preprint arXiv:2410.17555* (2024).

[5] Zhaoliang Chen, Zhihao Wu, Ylli Sadikaj, Claudia Plant, Hong-Ning Dai, Shiping Wang, Yiu-Ming Cheung, and Wenzhong Guo. 2024. ADEdgeDrop: Adversarial Edge Dropping for Robust Graph Neural Networks. *arXiv preprint arXiv:2403.09171* (2024).

[6] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2020. Adaptive universal generalized pagerank graph neural network. *arXiv preprint arXiv:2006.07988* (2020).

[7] Enyan Dai and Suhang Wang. 2020. Fairgnn: Eliminating the discrimination in graph neural networks with limited sensitive attribute information. *arXiv preprint arXiv:2009.01454* (2020).

[8] Enyan Dai and Suhang Wang. 2023. Learning Fair Graph Neural Networks With Limited and Private Sensitive Attribute Information. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2023), 7103–7117. doi:10.1109/TKDE.2022.3197554

[9] Enyan Dai, Tianxiang Zhao, Huaisheng Zhu, Junjie Xu, Zhimeng Guo, Hui Liu, Jiliang Tang, and Suhang Wang. 2024. A comprehensive survey on trustworthy graph neural networks: Privacy, robustness, fairness, and explainability. *Machine Intelligence Research* 21, 6 (2024), 1011–1061.

[10] Vishnu Asutosh Dasu, Ashish Kumar, Saeid Tizpaz-Niari, and Gang Tan. 2024. NeuFair: Neural Network Fairness Repair with Dropout. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1541–1553.

[11] Yushun Dong, Ninghao Liu, Brian Jalaian, and Jundong Li. 2022. Edits: Modeling and mitigating data bias for graph neural networks. In *Proceedings of the ACM web conference 2022*. 1259–1269.

[12] Zhan Gao, Subhrajit Bhattacharya, Leiming Zhang, Rick S Blum, Alejandro Ribeiro, and Brian M Sadler. 2021. Training robust graph neural networks with topology adaptive edge dropping. *arXiv preprint arXiv:2106.02892* (2021).

[13] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997* (2018).

[14] Sahin Cem Geyik, Stuart Ambler, and Krishnaram Kenthapadi. 2019. Fairness-aware ranking in search & recommendation systems with application to linkedin talent search. In *Proceedings of the 25th acm sigkdd international conference on knowledge discovery & data mining*. 2221–2231.

[15] Aparajita Haldar, Teddy Cunningham, and Hakan Ferhatosmanoglu. 2022. RAGUEL: Recourse-aware group unfairness elimination. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 666–675.

[16] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).

[17] Dongxiao He, Chundong Liang, Huixin Liu, Mingxiang Wen, Pengfei Jiao, and Zhiyong Feng. 2022. Block modeling-guided graph convolutional neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 4022–4029.

[18] Wen Huang, Lu Zhang, and Xintao Wu. 2022. Achieving counterfactual fairness for causal bandit. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 6952–6959.

[19] Zhimeng Jiang, Xiaotian Han, Chao Fan, Zirui Liu, Na Zou, Ali Mostafavi, and Xia Hu. 2024. Chasing Fairness in Graphs: A GNN Architecture Perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21214–21222.

[20] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.

[21] O Deniz Kose, Gonzalo Mateos, and Yanning Shen. 2024. Filtering as Rewiring for Bias Mitigation on Graphs. In *2024 IEEE 13rd Sensor Array and Multichannel Signal Processing Workshop (SAM)*. IEEE, 1–5.

[22] Peizhao Li, Yifei Wang, Han Zhao, Pengyu Hong, and Hongfu Liu. 2021. On dyadic fairness: Exploring and mitigating bias in graph connections. In *International conference on learning representations*.

[23] Manisha Padala and Sujit Gujar. 2020. Fnnc: Achieving fairness through neural networks. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence. IJCAI. https://www. ijcai. org/proceedings/2020/0315. pdf Go to original source.*

[24] Indro Spinelli, Simone Scardapane, Amir Hussain, and Aurelio Uncini. 2021. Fairdrop: Biased edge dropout for enhancing fairness in graph representation learning. *IEEE Transactions on Artificial Intelligence* 3, 3 (2021), 344–354.

[25] Ruijia Wang, Xiao Wang, Chuan Shi, and Le Song. 2022. Uncovering the structural fairness in graph contrastive learning. *Advances in neural information processing systems* 35 (2022), 32465–32473.

[26] Yu Wang, Yuying Zhao, Yushun Dong, Huiyuan Chen, Jundong Li, and Tyler Derr. 2022. Improving fairness in graph neural networks via mitigating sensitive attribute leakage. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 1938–1948.

[27] Cheng Yang, Jixi Liu, Yunhe Yan, and Chuan Shi. 2024. Fairsin: Achieving fairness in graph neural networks through sensitive information neutralization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 9241–9249.

[28] Guixian Zhang, Debo Cheng, Guan Yuan, and Shichao Zhang. 2024. Learning fair representations via rebalancing graph structure. *Information Processing & Management* 61, 1 (2024), 103570.

[29] Guixian Zhang, Debo Cheng, and Shichao Zhang. 2023. Fpgnn: Fair path graph neural network for mitigating discrimination. *World Wide Web* 26, 5 (2023), 3119–3136.

[30] Guixian Zhang, Guan Yuan, Debo Cheng, Lin Liu, Jiuyong Li, and Shichao Zhang. 2025. Disentangled contrastive learning for fair graph representations. *Neural Networks* 181 (2025), 106781.

[31] Shengzhong Zhang, Wenjie Yang, Yimin Zhang, Hongwei Zhang, Divin Yan, and Zengfeng Huang. 2023. Understanding Community Bias Amplification in Graph Representation Learning. *arXiv preprint arXiv:2312.04883* (2023).

[32] Tao Zhang, Tianqing Zhu, Mengde Han, Fengwen Chen, Jing Li, Wanlei Zhou, and Philip S Yu. 2023. Fairness in graph-based semi-supervised learning. *Knowledge and Information Systems* 65, 2 (2023), 543–570.

[33] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.